



India Space Academy, Department of Space Education



“From Fundamentals to Advanced Geospatial Intelligence: Summer Internship on Remote Sensing, GIS, Artificial Intelligence, and Python”

Programme Duration: 45 Days

Objective: The objective of this Summer Internship Programme is to equip participants with fundamental to advanced knowledge of Remote Sensing (RS), Geographic Information Systems (GIS), Artificial Intelligence (AI), and Python-based geospatial analysis, with a strong emphasis on practical and workflow-oriented problem solving. The Programme is structured to provide foundational understanding of geospatial concepts in the initial phase, followed by intensive hands-on training in Python for spatial data processing, analysis, and visualization. During the 15-day lecture phase, participants will develop skills in satellite data handling, raster and vector analysis, time-series interpretation, terrain modelling, and machine learning techniques using Python and GIS tools. This is followed by a 30-day guided project phase, where participants will apply these skills to advanced geospatial applications involving real-world datasets. Upon successful completion, participants will be able to independently design and execute geospatial workflows, perform spatial and temporal analysis, and develop AI-assisted solutions for environmental, urban, and disaster-related studies.

Tools/Software: QGIS, ArcGIS, Google Earth Engine (GEE), Python (Introductory), Google Colab/Jupyter Notebook, GPS, Open-Source Plugins

Mode: Online

Duration: 45 Days (2 hours/day – Live + Hands-on)

Platform: Google Meet/Zoom

Course Coordinator: Miss. Alisha Sinha

Pre-requisite: “Participants are not required to have prior experience in RS-GIS or AI; however, a strong motivation to learn and apply geospatial technologies is essential.”

Integrated Geospatial Workflow Design

Combining RS + GIS + Python + AI

- Designing complete project workflow
- Data integration and automation
- End-to-end workflow: Data → Processing → Analysis → Output → Interpretation

In the previous sessions, you were introduced to the basic concept of Artificial Intelligence and Machine Learning in geospatial applications. This is one of the most important sessions because real-world geospatial projects are not based on only one tool or one method.

Mini Project

Project Title: Integrated Wildfire Vulnerability Workflow using NDVI and Temperature

Datasets: We used Google Earth Engine only for **data preparation**.

We prepared three inputs for our Python wildfire workflow:

1. Madhya Pradesh boundary shapefile
2. NDVI raster
3. Temperature raster

After export, the main wildfire analysis will continue in Python.

Data Preparation in GEE

// Study Area: Madhya Pradesh

// Outputs: Boundary + NDVI + Temperature

Note: Remove // (double slash) from the codes before Run the script.

// 1. Load Madhya Pradesh boundary

//var india_states = ee.FeatureCollection("FAO/GAUL/2015/level1");

//var madhya_pradesh = india_states

// .filter(ee.Filter.eq("ADM0_NAME", "India"))

// .filter(ee.Filter.eq("ADM1_NAME", "Madhya Pradesh"));

//Map.centerObject(madhya_pradesh, 6);

//Map.addLayer(

```
// madhya_pradesh,
```

```
//{color: "black"},
```

```
// "Madhya Pradesh Boundary"
```

```
//);
```

```
// 2. Export Madhya Pradesh boundary as shapefile
```

```
//Export.table.toDrive({
```

```
// collection: madhya_pradesh,
```

```
// description: "Madhya_Pradesh_Boundary_Shapefile",
```

```
// folder: "GEE_Exports",
```

```
// fileNamePrefix: "Madhya_Pradesh_Boundary",
```

```
// fileFormat: "SHP"
```

```
//});
```

```
// 3. Prepare MODIS NDVI data
```

```
// MOD13Q1 NDVI: 16-day composite, 250 m
```

```
//var ndvi_collection = ee.ImageCollection("MODIS/061/MOD13Q1")
```

```
// .filterDate("2024-03-01", "2024-05-31")
```

```
// .filterBounds(madhya_pradesh)
```

```
// .select("NDVI");
```

```
//var ndvi = ndvi_collection
```

```
// .mean()
```

```
// .multiply(0.0001)
```

```
// .clip(madhya_pradesh);
```

```
//Map.addLayer(
```

```
// ndvi,
```

```
// {
```

```
// min: 0,
```

```
// max: 0.8,
```

```
// palette: ["brown", "yellow", "green"]
```

```
// },
```

```
// "Mean NDVI March-May 2024"
```

```
//);
```

```
// 4. Prepare MODIS Land Surface Temperature data
```

```
// MOD11A2 LST: 8-day composite, 1 km
```

```
//var lst_collection = ee.ImageCollection("MODIS/061/MOD11A2")
```

```
// .filterDate("2024-03-01", "2024-05-31")
```

```
// .filterBounds(madhya_pradesh)
```

```
// .select("LST_Day_1km");
```

```
//var temperature = lst_collection
```

```
// .mean()
```

```
// .multiply(0.02)
```

```
// .subtract(273.15)
```

```
// .clip(madhya_pradesh);
```

```

//Map.addLayer(

// temperature,

// {

// min: 25,

// max: 45,

// palette: ["blue", "yellow", "red"]

// },

// "Mean Temperature March-May 2024"

//);


// 5. Export NDVI raster


//Export.image.toDrive({

// image: ndvi,

// description: "MP_NDVI_Mar_May_2024",

// folder: "GEE_Exports",

// fileNamePrefix: "MP_NDVI_Mar_May_2024",

// region: madhya_pradesh.geometry(),

// scale: 250,

// crs: "EPSG:4326",

// maxPixels: 1e13

//});

```

// 6. Export Temperature raster

```
//Export.image.toDrive({  
// image: temperature,  
// description: "MP_Temperature_Mar_May_2024",  
// folder: "GEE_Exports",  
// fileNamePrefix: "MP_Temperature_Mar_May_2024",  
// region: madhya_pradesh.geometry(),  
// scale: 1000,  
// crs: "EPSG:4326",  
// maxPixels: 1e13  
});
```

Code Explanation:

```
/ Day 14 Mini Project  
// Data Preparation in GEE  
// Study Area: Madhya Pradesh  
// Outputs: Boundary + NDVI + Temperature
```

This is only a comment section. It tells us the purpose of the script.

1. Load Madhya Pradesh Boundary

```
var india_states = ee.FeatureCollection("FAO/GAUL/2015/level1");
```

This loads state-level administrative boundaries from the FAO GAUL dataset.

```
var madhya_pradesh = india_states  
  .filter(ee.Filter.eq("ADM0_NAME", "India"))  
  .filter(ee.Filter.eq("ADM1_NAME", "Madhya Pradesh"));
```

This filters only **Madhya Pradesh** from India.

2. Display Madhya Pradesh Boundary

```
Map.centerObject(madhya_pradesh, 6);
```

This zooms the map to Madhya Pradesh.

```
Map.addLayer(  
  madhya_pradesh,  
  {color: "black"},  
  "Madhya Pradesh Boundary"  
);
```

This displays the Madhya Pradesh boundary on the GEE map.

3. Export Madhya Pradesh Boundary

```
Export.table.toDrive({  
  collection: madhya_pradesh,  
  description: "Madhya_Pradesh_Boundary_Shapefile",  
  folder: "GEE_Exports",  
  fileNamePrefix: "Madhya_Pradesh_Boundary",  
  fileFormat: "SHP"  
});
```

This exports the Madhya Pradesh boundary as a shapefile to Google Drive.

4. Load MODIS NDVI Data

```
var ndvi_collection = ee.ImageCollection("MODIS/061/MOD13Q1")
```

This loads the MODIS NDVI dataset.

MOD13Q1 provides NDVI data at 250 m resolution.

```
.filterDate("2024-03-01", "2024-05-31")
```

This selects March to May 2024.

```
.filterBounds(madhya_pradesh)
```

This selects images covering Madhya Pradesh.

```
.select("NDVI");
```

This selects only the NDVI band.

5. Prepare Mean NDVI Raster

```
var ndvi = ndvi_collection  
  .mean()  
  .multiply(0.0001)  
  .clip(madhya_pradesh);
```

.mean() calculates the average NDVI for March–May 2024.

.multiply(0.0001) applies the MODIS scale factor and converts raw NDVI values into real NDVI values.

.clip(madhya_pradesh) cuts the raster only to Madhya Pradesh.

6. Display NDVI

```
Map.addLayer(  
  ndvi,  
  {  
    min: 0,  
    max: 0.8,  
    palette: ["brown", "yellow", "green"]  
  },  
  "Mean NDVI March-May 2024"  
);
```

This displays NDVI on the map.

Color meaning

Brown = low vegetation

Yellow = moderate vegetation

Green = healthy vegetation

7. Load MODIS Temperature Data

```
var lst_collection = ee.ImageCollection("MODIS/061/MOD11A2")
```

This loads MODIS Land Surface Temperature data.

*MOD11A2 provides LST at **1 km resolution**.*

```
.filterDate("2024-03-01", "2024-05-31")
```

This selects March to May 2024.

```
.filterBounds(madhya_pradesh)
```

This selects images covering Madhya Pradesh.

```
.select("LST_Day_1km");
```

This selects daytime land surface temperature.

8. Prepare Mean Temperature Raster

```
var temperature = lst_collection  
  .mean()  
  .multiply(0.02)  
  .subtract(273.15)  
  .clip(madhya_pradesh);
```

.mean() calculates average daytime LST for March–May.

.multiply(0.02) applies the MODIS LST scale factor.

.subtract(273.15) converts temperature from Kelvin to Celsius.

.clip(madhya_pradesh) cuts temperature data only to Madhya Pradesh.

9. Display Temperature

```
Map.addLayer(  
  temperature,  
  {  
    min: 25,  
    max: 45,  
    palette: ["blue", "yellow", "red"]  
  },  
  "Mean Temperature March-May 2024"  
);
```

This displays temperature on the map.

Color meaning

Blue = lower temperature

Yellow = moderate temperature

Red = higher temperature

10. Export NDVI Raster

```
Export.image.toDrive({  
  image: ndvi,  
  description: "MP_NDVI_Mar_May_2024",  
  folder: "GEE_Exports",  
  fileNamePrefix: "MP_NDVI_Mar_May_2024",  
  region: madhya_pradesh.geometry(),  
  scale: 250,  
  crs: "EPSG:4326",  
  maxPixels: 1e13  
});
```

This exports the NDVI raster to Google Drive.

*Important parameters

image: ndvi means export the NDVI image.
scale: 250 keeps MODIS NDVI at its native 250 m resolution.
region restricts export to Madhya Pradesh.
crs: "EPSG:4326" exports in WGS 84 coordinate system.

11. Export Temperature Raster

```
Export.image.toDrive({  
  image: temperature,  
  description: "MP_Temperature_Mar_May_2024",  
  folder: "GEE_Exports",
```

```

fileNamePrefix: "MP_Temperature_Mar_May_2024",
region: madhya_pradesh.geometry(),
scale: 1000,
crs: "EPSG:4326",
maxPixels: 1e13
});

```

This exports the temperature raster to Google Drive.

Important parameters

scale: 1000 is correct because MODIS LST is originally 1 km resolution.

Now the workflow will move from:

Google Earth Engine → Python

“How cloud-based Remote Sensing data becomes part of a Python GIS workflow.”

Important Workflow

Google Earth Engine
(Data Preparation)



Export Datasets



Python Workflow
(Analysis + Visualization)



Wildfire Vulnerability Understanding

Step 1: Import Required Libraries

Now type this code in the first Jupyter cell:

```

import geopandas as gpd
import rasterio
import matplotlib.pyplot as plt
import pandas as pd
import numpy as np

```

Press:

Shift + Enter

Code explanations

geopandas

Used for:

- shapefiles,
- GIS analysis,
- vector processing.

rasterio

Used for:

- raster reading,
- satellite image processing,
- NDVI and temperature analysis.

matplotlib.pyplot

Used for:

- plotting,
- maps,
- graphs,
- and visualization.

pandas

Used for:

- tables,
- CSV outputs,
- analytical datasets.

numpy

Used for:

- numerical operations,
- raster calculations,
- array processing.

Step 2: Define Dataset Paths

Now type this code in the next cell:

```
shapefile_path = r"C:\Users\ASUS\Desktop\ISA\2nd_Summer_Internship_Programme\Madhya Pradesh\Madhya_Pradesh_Boundary.shp"
```

```
ndvi_path = r"C:\Users\ASUS\Desktop\ISA\2nd_Summer_Internship_Programme\Madhya Pradesh\MP_NDVI_Mar_May_2024.tif"
```

```
temperature_path = r"C:\Users\ASUS\Desktop\ISA\2nd_Summer_Internship_Programme\Madhya Pradesh\MP_Temperature_Mar_May_2024.tif"
```

Press:

Shift + Enter

Why This Step Is Important

Python must know:

- where datasets are stored,
- and which files will be used in the workflow.

Step 3: Load the Shapefile

```
gdf = gpd.read_file(shapefile_path)
```

```
gdf.head()
```

Press:

Shift + Enter

Now Python has loaded the Madhya Pradesh boundary shapefile into a GeoDataFrame.

*Important GIS Understanding

A GeoDataFrame is similar to a table, but it also contains spatial geometry information.

Columns Explanation

ADM0_NAME: This represents the country name.

ADM1_NAME: This represents the state name.

Shape_Area: This stores the polygon area information.

Shape_Leng: This stores polygon boundary length or perimeter information.

geometry: This is the most important GIS column because it stores the actual spatial geometry.”

Without geometry:

- GIS analysis cannot happen.

“A normal table only stores attributes.

But a GeoDataFrame stores:

- attributes,
- plus spatial geometry together.”

Now the GIS boundary has successfully entered the Python workflow.

Next, we will connect:

- Remote Sensing raster data with:
- GIS polygon boundaries.

Step 5: Visualize Madhya Pradesh Boundary

```
gdf.plot(  
    figsize=(8,8),  
    color="lightgreen",  
    edgecolor="black"  
)  
  
plt.title("Madhya Pradesh Boundary")  
  
plt.show()
```

Press:

Shift + Enter

What are we doing here: “Before starting spatial analysis, we must visually verify whether the shapefile loaded correctly.”

Python will:

- display Madhya Pradesh boundary,
- visualize the GIS layer,
- and confirm successful loading.

Code Explanation

gdf.plot()

Plots the shapefile.

figsize=(8,8)

Controls map size.

color="lightgreen"

Gives polygon color.

edgecolor="black"

Displays polygon boundaries clearly.

Step 6: Load NDVI Raster

Now we are moving from GIS vector data toward Remote Sensing raster data.

Till now:

- we loaded the GIS boundary layer.

Now we will load:

NDVI raster data

This is where Remote Sensing and GIS start integrating together inside the workflow.

```
ndvi = rasterio.open(ndvi_path)
```

```
print(ndvi)
```

Press:

Shift + Enter

We are loading the satellite-derived NDVI raster into Python for analysis.

Python will:

- open the NDVI raster,
- read raster metadata,
- and prepare it for processing.

Code Explanation

`rasterio.open()`

Used to open raster datasets.

`ndvi_path`

Represents the NDVI raster location.

`print(ndvi)`

Displays raster information such as:

- raster type,
- file mode,
- and dataset path.

Raster datasets are fundamentally different from shapefiles.

Raster data stores information in:

- grid cells,
- or pixels.”

Each pixel in the NDVI raster contains vegetation information.

In wildfire studies:

- low NDVI may indicate stressed or dry vegetation,
- while higher NDVI usually represents healthier vegetation.

Now both major workflow components are available inside Python:

- GIS boundary,
- and Remote Sensing raster.”

Step 7: Read NDVI Pixel Values

```
ndvi_data = ndvi.read(1)

print(ndvi_data)
```

Press:

Shift + Enter

Why this step is important: Till now we only opened the raster file.

Now we are reading the actual pixel values stored inside the raster.

Code Explanation

read(1)

Reads:

Band 1

from the raster.

“Raster data is stored as numerical arrays. Every value represents a pixel.”

Step 8: Visualize NDVI Raster

```
plt.figure(figsize=(10,8))

plt.imshow(ndvi_data, cmap="YlGn")

plt.colorbar(label="NDVI")

plt.title("NDVI Raster - Madhya Pradesh")
```

```
plt.show()
```

Press:

Shift + Enter

“Raster datasets are best understood visually rather than by printing raw numerical arrays.

Code Explanation

```
plt.imshow()
```

Displays raster pixels as an image.

```
cmap="YlGn"
```

Uses Yellow-Green color palette suitable for vegetation visualization.

```
plt.colorbar()
```

Displays NDVI value scale.

Important NDVI Understanding

Color	Meaning
Yellow	Lower vegetation
Green	Healthier vegetation

“Areas with lower NDVI may represent:

- dry vegetation,
- sparse vegetation,
- or stressed vegetation,
which can become more wildfire-prone.”

Step 9: Load Temperature Raster

Now we will load the second important wildfire-related variable:

Temperature raster data.

Till now:

- we analyzed vegetation condition using NDVI.

Now we will integrate:

land surface temperature

because wildfire vulnerability is strongly influenced by heat conditions.

Code

```
temperature = rasterio.open(temperature_path)

print(temperature)
```

Press:

Shift + Enter

“We are loading satellite-derived land surface temperature data into Python.”

Python will:

- open the temperature raster,
- read raster metadata,
- and prepare it for analysis.

“Temperature is one of the most important environmental variables affecting wildfire occurrence.”

“Higher temperature increases:

- surface heat,
- vegetation dryness,
- and fire susceptibility.”

Variable	Purpose
NDVI	Vegetation condition
Temperature	Heat condition

Combining both variables helps understand: **wildfire vulnerability patterns**.

Step 10: Read Temperature Pixel Values

```
temperature_data = temperature.read(1)

print(temperature_data)
```

Press:

Shift + Enter

“Now we are reading the actual temperature pixel values stored inside the raster.”

“Each raster pixel stores: **land surface temperature information** for that geographic location.”

Step 11: Visualize Temperature Raster

```
plt.figure(figsize=(10,8))
```

```
plt.imshow(temperature_data, cmap="hot")
```

```
plt.colorbar(label="Temperature (°C) ")
```

```
plt.title("Land Surface Temperature - Madhya Pradesh")
```

```
plt.show()
```

Press:

Shift + Enter

“Visualization helps us understand the spatial temperature distribution across Madhya Pradesh.”

Code Explanation

plt.imshow()

Displays raster pixels as an image.

cmap="hot"

Uses a heat-style color palette suitable for temperature visualization.

plt.colorbar()

Displays the temperature scale.

Important Temperature Understanding

Color	Meaning
Dark colors	Lower temperature
Bright yellow/white	Higher temperature

“Higher temperature regions may experience:

- increased surface dryness,
- heat stress,
- and higher wildfire susceptibility.”

Step 13: Calculate Mean NDVI Value

```
mean_ndvi = np.nanmean(ndvi_data)
```

```
print("Mean NDVI Value:", mean_ndvi)
```

Press:

Here we are calculating the average vegetation condition across Madhya Pradesh.

Code Explanation

```
np.nanmean()
```

Calculates mean while ignoring:

NaN (NoData) values

In environmental raster analysis, NoData pixels must often be ignored during calculations. The mean NDVI provides a general understanding of vegetation health across the study area.

Step 14: Calculate Mean Temperature

```
mean_temperature = np.nanmean(temperature_data)
```

```
print("Mean Temperature:", mean_temperature)
```

Press:

Shift + Enter

Now we are calculating the average land surface temperature across Madhya Pradesh.

Step 15: Create a Simple Analytical Summary Table

```
summary_table = pd.DataFrame({  
    "Variable": ["Mean NDVI", "Mean Temperature"],  
    "Value": [mean_ndvi, mean_temperature]  
})
```

```
summary_table
```

Press:

Shift + Enter

“Geospatial workflows should generate structured analytical outputs such as tables, reports, and summaries.”
This step converts raster analysis into a readable analytical format.”

Step 16: Export the Analytical Summary Table

Now we will export our analytical results.

This is a very important stage because geospatial workflows should generate:

- final outputs,
- reports,
- and shareable analytical datasets.”

Code

```
summary_table.to_csv(
    "Wildfire_Summary_Table.csv",
    index=False
)

print("Summary table exported successfully")
```

Press:

Shift + Enter

Until now, the results existed only inside Python memory. Now we are saving the analytical output permanently as a CSV file.

Python will:

- create a CSV file,
- store the analytical table,
- and save it in the current working folder.

Code Explanation

to_csv()

Used to export tables into CSV format.

"Wildfire_Summary_Table.csv"

Represents the output filename.

index=False

Prevents extra row numbering column from being saved.

This is how analytical outputs are prepared for:

- reports,
- research papers,
- dashboards,
- decision support,
- and machine learning workflows.”

In real AI workflows, exported tables often become: **training datasets** for machine learning models.

Step 17: Create Wildfire Susceptibility Classes

```

susceptibility = np.where(
    (ndvi_data < 0.3) & (temperature_data > 40),
    2,
    np.where(
        (ndvi_data < 0.4) & (temperature_data > 35),
        1,
        0
    )
)

```

Press:

Shift + Enter

Susceptibility Classes

Value	Meaning
0	Low Risk
1	Moderate Risk
2	High Risk

We are creating wildfire susceptibility classes using environmental conditions.

Explanation

High Risk

$(ndvi_data < 0.3) \& (temperature_data > 40)$

Means:

low vegetation health
 high temperature
 higher wildfire vulnerability

Moderate Risk

$(ndvi_data < 0.4) \& (temperature_data > 35)$

Means:

moderate vegetation stress
 moderate heat stress

In real wildfire AI systems, susceptibility classes are usually created using:

historical fire occurrence,

field validation,
or fire inventory datasets.

Today we are creating simplified training labels only for workflow understanding.

***Error will create here because of different resolution of datasets**

Resample Temperature to Match NDVI

```
from rasterio.warp import reproject, Resampling

temperature_resampled = np.empty(
    ndvi_data.shape,
    dtype=np.float32
)

reproject(
    source=temperature_data,
    destination=temperature_resampled,
    src_transform=temperature.transform,
    src_crs=temperature.crs,
    dst_transform=ndvi.transform,
    dst_crs=ndvi.crs,
    resampling=Resampling.bilinear
)

print("NDVI shape:", ndvi_data.shape)
print("Original Temperature shape:", temperature_data.shape)
print("Resampled Temperature shape:", temperature_resampled.shape)
```

Here we are converting the temperature raster grid to match the NDVI raster grid. Now both datasets will have the same number of rows and columns, so we can combine them for wildfire susceptibility analysis.

Step 18: Create Wildfire Susceptibility Classes

```
susceptibility = np.where(
    (ndvi_data < 0.3) & (temperature_resampled > 40),
    2,
    np.where(
        (ndvi_data < 0.4) & (temperature_resampled > 35),
        1,
        0
    )
)
```

```
print("Susceptibility map created successfully")
```

```
print(np.unique(susceptibility))
```

Class meaning

Value	Meaning
0	Low susceptibility
1	Moderate susceptibility
2	High susceptibility

Step 19: Visualize the Wildfire Susceptibility Map

```
plt.figure(figsize=(10,8))
```

```
plt.imshow(  
    susceptibility,  
    cmap="YlOrRd"  
)
```

```
plt.colorbar(label="Wildfire Susceptibility Class")
```

```
plt.title("Simplified Wildfire Susceptibility Map")
```

```
plt.show()
```

Press:

Shift + Enter

Here we created susceptibility classes numerically, but numbers alone are difficult to interpret. Now we are converting the result into a map so students can visually understand spatial wildfire vulnerability patterns.

Code explanation

`plt.imshow()` displays the susceptibility raster.

`cmap="YlOrRd"` uses yellow-orange-red colors.

In this map:

Color tone	Meaning
Light yellow	Low susceptibility
Orange	Moderate susceptibility
Dark red	High susceptibility

`plt.colorbar()` adds the susceptibility class scale.

Step 20 : Prepare AI Training Dataset

Now we are moving toward the actual AI stage of the workflow. Machine Learning models cannot directly understand raster images. So first we must convert raster information into: structured numerical training data.

This is one of the most important stages in GeoAI workflows.

Our Machine Learning model will use:

NDVI,
and temperature

as predictor variables.

“The susceptibility classes we created earlier will act as:
training labels.”

Variable	Type	Role
NDVI	Predictor	feature
Temperature	Predictor	feature
Susceptibility	Target	label

Step 21: Convert Raster Data into AI Dataset

```
X = np.column_stack((  
    ndvi_data.flatten(),  
    temperature_resampled.flatten()  
))
```

```
y = susceptibility.flatten()
```

```
print("Feature shape:", X.shape)  
print("Target shape:", y.shape)
```

Press:

Shift + Enter

“Machine Learning models require:

tabular numerical input

not raster grids.”

.flatten() Raster data is originally stored as 2D arrays.

Flatten converts them into:

one-dimensional vectors.

column_stack() combines:

NDVI, and temperature into one AI feature matrix.”

Step 22: Remove Invalid Pixel Values

Before training Machine Learning models, we must clean the dataset.

Environmental raster datasets often contain:

NaN values,
NoData values,
or invalid pixels.

If we directly train AI models using invalid values, the model may fail or produce incorrect results.

Machine Learning algorithms require:
clean numerical datasets.”

Remove NaN Values

```
valid_mask = (  
    ~np.isnan(X[:,0]) &  
    ~np.isnan(X[:,1]) &  
    ~np.isnan(y)  
)
```

```
X_valid = X[valid_mask]  
y_valid = y[valid_mask]
```

```
print("Clean feature shape:", X_valid.shape)  
print("Clean target shape:", y_valid.shape)
```

Press:

Code Explanation**np.isnan()**

Checks for:

invalid NoData values

valid_mask : Creates a filter selecting only:

valid NDVI,

valid temperature,

and valid susceptibility pixels.

X_valid: Stores cleaned predictor variables.

y_valid: Stores cleaned susceptibility labels.

Data cleaning is one of the most important stages in AI workflows because model quality strongly depends on input data quality. In real environmental AI systems, large amounts of preprocessing and cleaning are required before model training.

Step 23: Split Training and Testing Data

Before training Machine Learning models, we divide the dataset into:

training data,

and testing data.

The model learns from training data and is evaluated using testing data. This helps evaluate whether the model can generalize properly to unseen data.

Code

```
from sklearn.model_selection import train_test_split
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X_valid,
    y_valid,
    test_size=0.3,
    random_state=42
)
```

```
print("Training samples:", X_train.shape)
```

```
print("Testing samples:", X_test.shape)
```

Press:

Shift + Enter

Code Explanation

`train_test_split()`

Splits data into:

training set,
and testing set.
`test_size=0.3`

Means:

70% training
30% testing
`random_state=42`

Ensures reproducible results.

Machine Learning models must always be evaluated on unseen data to check their predictive performance.

Step 24: Train the Random Forest Model

Before prediction, the AI model must first complete training. Training means the model learns the relationship between:

NDVI,
temperature,
and wildfire susceptibility classes.

```
from sklearn.ensemble import RandomForestClassifier
```

```
rf_model = RandomForestClassifier(  
    n_estimators=50,  
    random_state=42  
)
```

```
rf_model.fit(X_train, y_train)
```

```
print("Random Forest model trained successfully")
```

Press:

Shift + Enter

Now the Random Forest model is learning environmental wildfire relationships from the training dataset.

Machine Learning models must always be trained before prediction can happen.

Step 24: Predict Wildfire Susceptibility Using Random Forest

Now the Random Forest model has completed training.

The next step is **prediction**.

Now the AI model will predict wildfire susceptibility classes using environmental conditions.

```
rf_prediction = rf_model.predict(X_test)
print("Random Forest prediction completed")
```

Press:

Shift + Enter

The trained Random Forest model is now predicting:

- low susceptibility,
- moderate susceptibility,
- and high susceptibility

for unseen testing data.

This is the actual predictive AI stage of the workflow where environmental patterns learned during training are used for wildfire susceptibility classification.”

Step 25: Evaluate Random Forest Model Accuracy

```
from sklearn.metrics import accuracy_score, classification_report
rf_accuracy = accuracy_score(y_test, rf_prediction)
print("Random Forest Accuracy:", rf_accuracy)

print("\nClassification Report:\n")
print(classification_report(y_test, rf_prediction))
```

Prediction alone is not enough. We must check how accurately the model predicted wildfire susceptibility classes.

Code explanation

`accuracy_score()` calculates the overall accuracy of the Random Forest model.

`classification_report()` gives class-wise:

- precision
- recall
- F1-score
- support

Step 26 : Create Random Forest Susceptibility Map

```
rf_full_prediction = rf_model.predict(X_valid)
```

```
rf_susceptibility_map = np.full(
    susceptibility.shape,
    np.nan
)
```

```
rf_susceptibility_map[valid_mask.reshape(susceptibility.shape)] = rf_full_prediction
```

```
plt.figure(figsize=(10,8))
```

```
plt.imshow(
    rf_susceptibility_map,
    cmap="YlOrRd"
)
```

```
plt.colorbar(label="RF Wildfire Susceptibility Class")
```

```
plt.title("Random Forest Wildfire Susceptibility Map")
```

```
plt.show()
```

We are converting Random Forest predictions back into map form. Now the trained Random Forest model is applied to all valid pixels and converted back into a spatial wildfire susceptibility map. This is the final AI-based wildfire susceptibility output.”

Step 27: Feature Importance Analysis

Now the Random Forest model has successfully generated the wildfire susceptibility prediction.

But an important scientific question still remains: which environmental variable contributed more toward wildfire susceptibility prediction?

To answer this, we perform: Feature Importance Analysis.”

Machine Learning models can generate predictions, but researchers also need to understand:

which variables influence prediction most strongly,
and how environmental factors contribute to wildfire susceptibility.”

Feature importance is one of the most useful outputs of Random Forest because it helps interpret the AI model scientifically.

Code

```
importance = rf_model.feature_importances_
```

```
feature_table = pd.DataFrame({  
    "Feature": ["NDVI", "Temperature"],  
    "Importance": importance  
})
```

```
feature_table
```

Press:

Shift + Enter

Code Explanation

feature_importances_ This function extracts:

variable contribution scores from the trained Random Forest model.

What Is Happening Here?

“The Random Forest model internally calculates how strongly each variable contributes toward wildfire susceptibility prediction.”

pd.DataFrame()

Creates a structured analytical table showing:

variable names, and their importance scores.

Output

Variable	Importance
NDVI	0.648
Temperature	0.352

Interpretation of the Result

“NDVI contributed approximately:

65%

toward wildfire susceptibility prediction.”

This indicates:

vegetation condition,
vegetation stress,
and fuel condition

played the dominant role in the model.”

Temperature Importance = 0.352

Temperature contributed approximately:

35%

toward wildfire susceptibility prediction.”

This means:

heat stress also influenced wildfire susceptibility,
but less strongly compared to vegetation condition.”

Important Scientific Understanding

This does not mean temperature is unimportant.

Instead, it means:

vegetation condition contributed more strongly

within this specific simplified workflow.

Step 28: Visualize Feature Importance

```
plt.figure(figsize=(6,5))
```

```
plt.bar(  
    feature_table["Feature"],  
    feature_table["Importance"]  
)
```

```
plt.title("Random Forest Feature Importance")
```

```
plt.ylabel("Importance Score")
```

```
plt.show()
```

Press:

Shift + Enter

Visualization helps us compare environmental variable influence more clearly.

Python will generate a:

feature importance bar chart

showing:

NDVI contribution,
and temperature contribution.

Code Explanation

plt.bar(): Creates the feature importance bar graph.

Feature

Represents predictor variables.

If NDVI bar is higher:

“Vegetation condition contributed more strongly toward wildfire susceptibility prediction.”

If Temperature bar is higher:

“Temperature contributed more strongly toward wildfire susceptibility prediction.”

Today’s session demonstrated a complete integrated GeoAI workflow for simplified wildfire susceptibility mapping. We combined Remote Sensing, GIS, Python, environmental analysis, and Artificial Intelligence together into one operational workflow. The Random Forest model identified NDVI as the dominant environmental predictor, indicating that vegetation condition

and fuel characteristics played the strongest role in wildfire susceptibility prediction within this study.